

A Practical Guide to Sparse attention transformer

TOPIC -- SPARSE ATTENTION TRANSFORMER

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

-- 01 --

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the next token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.

-- 02 --

Seq2seq models with attention (including self-attention) still suffered from the same issue with recurrent networks, which is that they are hard to parallelize, which prevented them from being accelerated on GPUs. In 2016, decomposable attention applied a self-attention mechanism to feedforward networks, which are easy to parallelize, and achieved SOTA result in textual entailment with an order of magnitude fewer parameters than LSTMs. One of its authors, Jakob Uszkoreit, suspected that attention without recurrence would be sufficient for language translation, thus the title "attention is all you need". That hypothesis was against conventional wisdom at the time, and even his father Hans Uszkoreit, a well-known computational linguist, was skeptical. In the same year, self-attention (called intra-attention or intra-sentence attention) was proposed for LSTMs. On 2017-06-12, the original (100M-parameter) encoder-decoder transformer model was published in the "Attention is all you need" paper. At the time, the focus of the research was on improving seq2seq for machine translation, by removing its recurrence to process all tokens in parallel, but preserving its dot-product attention mechanism to keep its text processing performance. This led to the introduction of a multi-head attention model that was easier to parallelize due to the use of independent heads and the lack of recurrence. Its parallelizability was an important factor to its widespread use in large neural networks.

-- 03 --

In deep learning, the transformer is a family of artificial neural network architectures based on the multi-head attention mechanism, in which text is converted to numerical representations called tokens, and each token is converted into a vector via lookup from a word embedding table. At each layer, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished. Because self-attention alone is permutation-invariant, transformers inject positional information, typically through positional encodings or learned positional embeddings, so token order can affect the output. Transformers have the advantage of having no recurrent units, therefore requiring less training time than earlier recurrent neural architectures (RNNs) such as long short-term memory (LSTM). Later variations have been widely adopted for training large language models (LLMs) on large (language) datasets. Modern transformer designs are commonly grouped into encoder-only, decoder-only, and encoder-decoder variants, depending on whether they are optimized for representation learning, autoregressive generation, or conditional sequence-to-sequence tasks.

-- 04 --

Un-embedding An un-embedding layer is almost the reverse of an embedding layer. Whereas an embedding layer converts a token identifier into a vector, an un-embedding layer converts a vector into a probability distribution over tokens.

-- 05 --

In general, there are 3 classes of language modelling tasks: "masked", "autoregressive", and "prefixLM". These classes are independent of a specific modeling architecture such as transformer, but they are often discussed in the context of transformer. In a masked task, one or more of the tokens is masked out, and the model would produce a probability distribution predicting what the masked-out tokens are based on the context. The loss function for the task is typically sum of log-perplexities for the masked-out tokens: $\text{Loss} = - \sum_{t \in \text{masked tokens}} \ln(\text{probability of } t \text{ conditional on its context})$ and the model is trained to minimize this loss function. The BERT series of models are trained for masked token prediction and another task. In an autoregressive task, the entire sequence is masked at first, and the model produces a probability distribution for the first token. Then the first token is revealed and the model predicts the second token, and so on.

The loss function for the task is still typically the same. The GPT series of models are trained by autoregressive tasks. In a prefixLM task, the sequence is divided into two parts. The first part is presented as context, and the model predicts the first token of the second part. Then that would be revealed, and the model predicts the second token, and so on. The loss function for the task is still typically the same. The T5 series of models are trained by prefixLM tasks. Note that "masked" as in "masked language modelling" is not "masked" as in "masked attention", and "prefixLM" as in "prefix language modeling" is not "prefixLM" as in "prefix language model".

-- 06 --

The original version of the transformer architecture was proposed in the 2017 paper "Attention Is All You Need" by researchers at Google. The predecessors of transformers were developed as an improvement over previous architectures for machine translation, but have found many applications since. They are used in large-scale natural language processing, computer vision (vision transformers), reinforcement learning, audio, multimodal learning, robotics,, playing chess and at disaster responses. It has also led to the development of pre-trained systems, such as generative pre-trained transformers (GPTs) and BERT (bidirectional encoder representations from transformers).

-- 07 --

```
/* first sublayer */ z_e_copy ← copy(z_e) for each t in 1:length(z_e) do z_e[t] ←
layer.layer_norm(z_e[t]) z_e ← layer.multihead_attention(z_e, z_e, z_e) for each
t in 1:length(z_e) do z_e[t] ← z_e[t] + z_e_copy[t]
```

-- 08 --

The attention mechanism used in the transformer architecture are scaled dot-product attention units. For each unit, the transformer model learns three weight matrices: the query weights W^Q , the key weights W^K , and the value weights W^V . The module takes three sequences, a query sequence, a key sequence, and a value sequence. The query sequence is a sequence of length $\ell_{\text{seq, query}}$, and each entry is a vector of dimension $d_{\text{emb, query}}$. Similarly for the key and value sequences. For each vector x_i in the query sequence, it is multiplied by a matrix W^Q to produce a query vector $q_i = x_i W^Q$. The matrix of all query vectors is the query matrix: $Q = X_{\text{query}} W^Q$. Similarly, we construct the key matrix K

$K = X_{\text{key}} W^K$ and the value matrix $V = X_{\text{value}} W^V$. It is usually the case that all W^Q, W^K, W^V are square matrices, meaning $d_{\text{emb}}, d_{\text{query}} = d_{\text{key}} = d_{\text{value}}$, etc. Attention weights are calculated using the query and key vectors: the attention weight a_{ij} from token i to token j is the dot product between q_i and k_j . The attention weights are divided by the square root of the dimension of the key vectors, d_k , which stabilizes gradients during training, and passed through a softmax which normalizes the weights. The fact that W^Q and W^K are different matrices allows attention to be non-symmetric: if token i attends to token j (i.e. $q_i \cdot k_j$ is large), this does not necessarily mean that token j will attend to token i (i.e. $q_j \cdot k_i$ could be small). The output of the attention unit for token i is the weighted sum of the value vectors of all tokens, weighted by a_{ij} , the attention from token i to each token. The attention calculation for all tokens can be expressed as one large matrix calculation using the softmax function, which is useful for training due to computational matrix operation optimizations that quickly compute matrix operations. The matrices Q, K and V are defined as the matrices where the i th rows are vectors q_i, k_i , and v_i respectively. Then we can represent the attention as $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$